Git Branching, Merging, Tags, Rebasing & Undoing Changes — Detailed Notes

1. What is a Branch in Git?

A branch is a separate line of development in a Git repository.

Think of your project like this:

```
main
  |
  A---B---C
           \
            D---E   feature-login
```

- `main` → main/stable version of the project
- `feature-login` → branch where you develop the login feature
- Changes in the feature branch do not immediately affect `main`.

Why use branches?

Branches allow developers to:

- Work on new features
- Fix bugs
- Experiment safely
- Work with other developers
- Keep the main project stable

2. Creating a Branch

Command

```
git branch branch-name
```

Example:

```
git branch feature-login
```

This creates a branch called feature-login.

Important

Creating a branch does not switch you to that branch.

For example:

```
git branch feature-login
```

creates the branch, but you are still on your previous branch

3. Creating and Switching to a Branch

You can create and switch to a branch in one command.

Modern command

```
git switch -c feature-login
```

Example:

```
git switch -c feature-login
```

Now you are working on:

```
feature-login
```

Older command

You may also see:

```
git checkout -b feature-login
```

Both accomplish the same basic task.

4. Switching Between Branches

To switch to an existing branch:

```
git switch branch-name
```

Example:

```
git switch main
```

To switch back:

```
git switch feature-login
```

Using checkout

Older Git tutorials commonly use:

```
git checkout main
```

and:

```
git checkout feature-login
```

git switch is generally clearer because it is specifically designed for switching branches.

5. Viewing Branches

To see local branches:

```
git branch
```

Example output:

```
  feature-login
* main
```

The * indicates the branch you are currently using.

Here:

```
* main
```

means you are currently on main.

View Remote Branches

```
git branch -r
```

Example:

```
origin/main
origin/feature-login
```

View All Branches

```
git branch -a
```

Example:

```
* main
  feature-login
  bug-fix
  remotes/origin/main
  remotes/origin/feature-login
```

6. Practical Example — Creating a Feature Branch

Suppose you have a website:

```
student-website
```

You want to add a login page.

First check your branch:

```
git branch
```

Suppose you are on:

```
* main
```

Create a new branch:

```
git switch -c login-page
```

Now:

```
git branch
```

Output:

```
* login-page
  main
```

You can now safely work on the login page.

7. Making Changes on a Branch

Suppose you create:

```
login.html
```

Check the status:

```
git status
```

Add the file:

```
git add login.html
```

Commit it:

```
git commit -m "Add login page"
```

Now the login-page branch contains the login-page changes.

8. Merging Branches

Merging means combining changes from one branch into another branch.

For example:

```
main
  A---B
       \
        C---D
            login-page
```

You want to bring the login-page changes into main.

First switch to main:

```
git switch main
```

Then merge:

```
git merge login-page
```

Git combines the changes.

9. Important Rule for Merging

Always switch to the branch that should receive the changes.

For example:

```
git switch main
git merge login-page
```

Means:

Take the changes from `login-page` and merge them into `main`.

Not:

```
git switch login-page
git merge main
```

unless you specifically want to bring main changes into login-page.

10. Fast-Forward Merge

A fast-forward merge happens when the target branch has not moved forward since the feature branch was created.

Example:

```
Before:

A---B---C
    main
         \
          D---E
              feature
```

Actually, conceptually the branches are:

```
A---B---C---D---E
    main       feature
```

If main is still pointing at B:

```
A---B---C---D---E
    main       feature
```

Git can simply move the main pointer forward.

After:

```
A---B---C---D---E
                main
                feature
```

There is no additional merge commit.

Example

```
git switch main
git merge feature-login
```

Git may display:

```
Fast-forward
```

11. Why is it Called Fast-Forward?

Git doesn't need to create a new commit.

It simply moves the branch pointer forward.

Think of it as:

```
main
  ↓
A---B
     \
      C---D feature
```

After merge:

```
A---B---C---D
             ↑
          main/feature
```

12. Three-Way Merge

A three-way merge occurs when both branches have new commits after they separated.

Example:

```
        C---D feature
       /
A---B
       \
        E---F main
```

Here:

- `B` = common ancestor
- `D` = latest commit on feature
- `F` = latest commit on main

Git compares three points:

1. 1. Common ancestor (`B`)
2. 2. Current `main` (`F`)
3. 3. Current feature (`D`)

Therefore it is called a three-way merge.

Example

Suppose:

```
main:
A---B---C

feature:
    \
     D---E
```

Both branches have changed.

You run:

```
git switch main
git merge feature
```

Git may create:

```
A---B---C-------M
     \         /
      D---E----
```

M is the merge commit.

13. Fast-Forward vs Three-Way Merge

	Fast-Forward	Three-Way Merge	
	No merge commit required	Merge commit usually created	
	Target branch has not diverged	Both branches have new commits	
	Branch pointer moves forward	Git combines two histories	
	Simple history	More complex history	

Easy way to remember

Fast-forward:

"Only one side moved."

Three-way merge:

"Both sides moved.

14. Merge Conflicts

A merge conflict occurs when Git cannot automatically determine which change should be kept.

For example, suppose main contains:

```
Hello Student
```

and your feature branch changes the same line to:

```
Hello Developer
```

At the same time, main changes it to:

```
Hello Everyone
```

Git doesn't know which version you want.

Therefore:

```
CONFLICT
```

15. How a Conflict Looks

Git may place conflict markers inside the file:

```
<<<<<<< HEAD
Hello Everyone
=======
Hello Developer
>>>>>>> feature-login
```

Meaning:

```
<<<<<<< HEAD
```

starts the current branch version.

```
=======
```

separates the two versions.

```
>>>>>>> feature-login
```

shows the incoming branch version.

16. Resolving a Merge Conflict

Suppose you want to keep:

```
Hello Developer
```

Edit the file and remove the conflict markers.

Final file:

```
Hello Developer
```

Then check:

```
git status
```

Add the resolved file:

```
git add filename
```

Then complete the merge:

```
git commit
```

or, depending on Git's state:

```
git commit -m "Resolve merge conflict"
```

17. Complete Conflict Resolution Example

Start with:

```
git switch main
```

Merge:

```
git merge feature-login
```

Git reports a conflict.

Check:

```
git status
```

Open the conflicting file.

Resolve the conflict.

Then:

```
git add .
```

Complete the merge:

```
git commit -m "Resolve merge conflict"
```

18. Cancelling a Merge

If the conflict becomes complicated and you want to cancel the merge:

```
git merge --abort
```

This attempts to return the working tree to the state before the merge started.

19. Deleting a Branch

After successfully merging a feature branch, you may no longer need it.

Delete a local branch:

```
git branch -d feature-login
```

The -d option means delete the branch safely.

Git normally prevents deletion if the branch contains unmerged work.

Force Delete a Branch

```
git branch -D feature-login
```

-D forces deletion.

Be careful!

Use -D only when you are sure you don't need the unmerged work.

20. Deleting a Remote Branch

To delete a branch from the remote repository:

```
git push origin --delete feature-login
```

Example:

```
git push origin --delete login-page
```

This removes the branch from the remote repository such as GitHub.

21. Git Tags

A tag is a name given to a specific commit.

Tags are commonly used for:

- Software releases
- Versions
- Milestones

For example:

```
v1.0
v2.0
v2.1
```

Think of a tag as a label attached to a commit.

22. Creating a Tag

Simple tag:

```
git tag v1.0
```

This creates a tag named:

```
v1.0
```

View Tags

```
git tag
```

Example:

```
v1.0
v1.1
v2.0
```

23. Annotated Tags

A more descriptive tag can be created using:

```
git tag -a v1.0 -m "Version 1.0 release"
```

Here:

- `-a` → annotated tag
- `v1.0` → tag name
- `-m` → message

24. Tag a Previous Commit

First view commits:

```
git log --oneline
```

Example:

```
8f42abc Add payment page
6d32def Add login page
4a21cde Initial website
```

You can tag a specific commit:

```
git tag v1.0 6d32def
```

Now v1.0 points to commit 6d32def.

25. Pushing Tags to GitHub

Creating a tag locally does not automatically send it to GitHub.

Push one tag:

```
git push origin v1.0
```

Push all tags:

```
git push origin --tags
```

26. Rebasing Basics

Rebase is another way to integrate changes from one branch into another.

Suppose:

```
        C---D feature
       /
A---B
       \
        E---F main
```

You want your feature branch to use the latest main.

You can run:

```
git switch feature
git rebase main
```

Git takes your feature commits and places them on top of the latest main.

Conceptually:

```
Before:

        C---D feature
       /
A---B---E---F main
```

After rebase:

```
A---B---E---F---C'---D'
                    feature
```

The commits may receive new commit IDs because Git is recreating them on a new base.

27. Why Use Rebase?

Rebase can create a cleaner, more linear history.

Instead of:

```
A---B---E---F
     \
      C---D
```

you get:

```
A---B---E---F---C'---D'
```

This can make the project history easier to read.

28. Rebase vs Merge

Merge

```
git switch main
git merge feature
```

Preserves the branching history.

Rebase

```
git switch feature
git rebase main
```

Moves/replays feature commits on top of the latest main.

Important Warning

Avoid rebasing commits that other people are already depending on, especially shared/public branches, because rebase rewrites commit history.

29. Undoing Changes in Git

Git provides several ways to undo changes.

The three important commands are:

```
git reset
git revert
git checkout
```

Modern Git also uses:

```
git restore
```

for many file-level restoration tasks.

The important thing is to understand what exactly you want to undo.

30. `git reset`

git reset moves the current branch pointer to another commit.

Example history:

```
A---B---C---D
```

Suppose you want to remove commit D from the current branch history.

You can use:

```
git reset --hard C
```

Result:

```
A---B---C
```

The branch pointer moves from D to C.

31. Types of Reset

There are three important reset modes.

Soft Reset

```
git reset --soft HEAD~1
```

Removes the last commit but keeps the changes staged.

Useful when:

"I committed too early and want to modify the commit."

Mixed Reset

```
git reset HEAD~1
```

or:

```
git reset --mixed HEAD~1
```

Removes the commit and unstages the changes, but keeps the files modified.

Hard Reset

```
git reset --hard HEAD~1
```

Removes the commit and discards the associated changes from the working directory.

Be careful with --hard.

32. Understanding `HEAD~1`

You will often see:

```
HEAD~1
```

It means:

The commit immediately before the current commit.

For example:

```
A---B---C
        ↑
       HEAD
```

Then:

```
HEAD~1 = B
```

And:

```
HEAD~2 = A
```

33. `git revert`

git revert creates a new commit that reverses the changes of an earlier commit.

Example:

```
A---B---C
```

Suppose C introduced a bug.

Run:

```
git revert C
```

Git creates a new commit:

```
A---B---C---D
```

D reverses the changes made by C.

Important Difference

reset:

Moves the branch backward.

revert:

Creates a new commit that undoes an earlier commit.

34. Why Revert is Safer for Shared Branches

Suppose you already pushed:

```
A---B---C
```

to GitHub.

Instead of changing the shared history with reset, you can use:

```
git revert C
```

Result:

```
A---B---C---D
```

Everyone can see the history clearly.

This is why revert is commonly preferred for undoing changes on shared branches such as main.

35. `git checkout`

git checkout has historically been used for multiple purposes.

Switch branch

```
git checkout main
```

Create and switch branch

```
git checkout -b feature-login
```

Restore an old version of a file

Older Git syntax:

```
git checkout -- index.html
```

This discards uncommitted changes to index.html and restores the version from the current commit.

Modern alternative

Use:

```
git restore index.html
```

This is easier to understand because Git separates branch switching (switch) from file restoration (restore).

36. Reset vs Revert vs Checkout

Command	Main Purpose
git reset	Move branch/HEAD backward
git revert	Create a new commit that undoes another commit
git checkout	Older command for switching branches and restoring files
git switch	Modern branch switching
git restore	Modern file restoration

37. Complete Practical Example

Imagine you are building a student website.

Initial project:

```
A---B
    main
```

Create a feature branch:

```
git switch -c feature-homepage
```

Create home.html.

Then:

```
git add home.html
git commit -m "Add homepage"
```

History:

```
A---B---C
    main  feature-homepage
```

Switch to main:

```
git switch main
```

Merge:

```
git merge feature-homepage
```

If main has not changed, Git performs a fast-forward merge:

```
A---B---C
        ↑
     main
     feature
```

Delete the feature branch:

```
git branch -d feature-homepage
```

38. Practical Merge Conflict Example

Suppose index.html contains:

```
<h1>Welcome Students</h1>
```

On main, someone changes it to:

```
<h1>Welcome to Our University</h1>
```

On your feature branch, you change the same line to:

```
<h1>Welcome Developers</h1>
```

When you merge:

```
git switch main
git merge feature
```

Git may report:

```
CONFLICT
```

The file could contain:

```
<<<<<<< HEAD
<h1>Welcome to Our University</h1>
=======
<h1>Welcome Developers</h1>
>>>>>>> feature
```

You decide the final version should be:

```
<h1>Welcome to Our University</h1>
```

Remove the conflict markers.

Then:

```
git add index.html
git commit -m "Resolve homepage merge conflict"
```

Conflict solved.

39. A Simple Git Workflow to Remember

For normal feature development:

```
1. Start from main
       ↓
```

```
2. Create a branch
       ↓
3. Make changes
       ↓
4. git add
       ↓
5. git commit
       ↓
6. Switch to main
       ↓
7. Merge feature branch
       ↓
8. Delete feature branch
```

Commands:

```
git switch main
git pull

git switch -c feature-login

# make changes

git add .
git commit -m "Add login feature"

git switch main
git merge feature-login

git branch -d feature-login
```

40. Quick Command Cheat Sheet

Branches

```
git branch
```

View branches.

```
git branch feature-login
```

Create branch.

```
git switch feature-login
```

Switch branch.

```
git switch -c feature-login
```

Create + switch.

Merge

```
git switch main
git merge feature-login
```

Merge feature into main.

```
git merge --abort
```

Cancel an ongoing merge.

Delete

```
git branch -d feature-login
```

Delete local branch safely.

```
git branch -D feature-login
```

Force delete local branch.

```
git push origin --delete feature-login
```

Delete remote branch.

Tags

```
git tag
```

View tags.

```
git tag v1.0
```

Create tag.

```
git tag -a v1.0 -m "Version 1.0"
```

Create annotated tag.

```
git push origin v1.0
```

Push tag.

```
git push origin --tags
```

Push all tags.

Rebase

```
git switch feature
git rebase main
```

Rebase feature onto main.

Undo

```
git reset --soft HEAD~1
```

Undo commit, keep changes staged.

```
git reset --mixed HEAD~1
```

Undo commit, keep changes unstaged.

```
git reset --hard HEAD~1
```

Undo commit and discard changes.

```
git revert <commit-id>
```

Create a new commit that reverses an earlier commit.

```
git restore filename
```

Discard uncommitted changes to a file.

41. Easy Memory Trick

Remember Git operations like this:

BRANCH

Create a separate workspace.

SWITCH

Move to another workspace.

MERGE

Bring two lines of work together.

FAST-FORWARD

Just move the branch pointer forward.

THREE-WAY MERGE

Both branches changed → Git combines their histories.

CONFLICT

Git doesn't know which change to keep → you decide.

TAG

Give an important commit a version label.

REBASE

Put your work on top of the latest base.

RESET

Move the branch pointer backward.

REVERT

Make a new commit that undoes an old commit.

CHECKOUT / RESTORE

Older `checkout` can switch branches or restore files; modern Git separates these into `switch` and `restore`.

byteXL

Shoolini University

Git Branching, Merging, Tags, Rebasing and Undoing Changes – Exercises

Exercise 1: Git Branching and Basic Merge — With Answer

Scenario

You are working on a project called Student Portal. You need to create a new feature branch, add a login feature, merge it into the main branch, and delete the feature branch.

Tasks

1. Create a folder named student-portal.
2. Initialize a Git repository.
3. Create a file named README.md.
4. Add the text: Student Portal Project
5. Commit the file with the message: Initial project setup
6. Create a branch named feature-login.
7. Switch to the feature-login branch.
8. Create a file named login.txt.
9. Add the text: Login Feature Added
10. Check Git status.
11. Add and commit the file with the message: Add login feature.
12. Switch back to main.
13. Merge feature-login into main.
14. Delete the feature-login branch safely.
15. View the final Git history.

Answer

```
mkdir student-portal
cd student-portal
git init

echo "Student Portal Project" > README.md
git add README.md
git commit -m "Initial project setup"

git branch feature-login
git switch feature-login
git branch

echo "Login Feature Added" > login.txt
git status
git add login.txt
git commit -m "Add login feature"

git switch main
git merge feature-login
```

```
git branch -d feature-login
git log --oneline --graph --all
```

Key Point

The * symbol in git branch indicates the currently active branch.

Exercise 2: Complete Git Practice Challenge

Scenario

Create a project called student-management-system. Complete all tasks yourself without looking at the answers.

Task 1: Initialize Repository

1. Create student-management-system.
2. Initialize Git.
3. Create README.md.
4. Add: Student Management System
5. Commit with: Initial project setup.

Task 2: Student Feature

1. Create and switch to feature-student.
2. Create students.txt.
3. Add: Student Registration Module
4. Commit with: Add student module.

Task 3: Course Feature

1. Switch to main.
2. Create and switch to feature-course.
3. Create courses.txt.
4. Add: Course Management Module
5. Commit with: Add course module.

Task 4: Result Feature

1. Switch to main.
2. Create and switch to feature-result.
3. Create results.txt.
4. Add: Student Result Module
5. Commit with: Add result module.

Task 5: Merge All Branches

Switch to main and merge feature-student, feature-course, and feature-result.

Task 6: Create a Merge Conflict

Create message.txt on main with: Welcome Students.
Commit it.
Create feature-message and change the same line.
Switch back to main and change the same line differently.
Merge the feature branch, resolve the conflict, and complete the merge.

Task 7: Create Tags

1. Create a lightweight tag v1.0.
2. Create an annotated tag v1.1.
3. View all tags.
4. Push one tag.
5. Push all tags.

Task 8: Rebase

1. Create feature-dashboard.
2. Add dashboard.txt and commit it.
3. Make another commit on main.
4. Switch back to feature-dashboard.
5. Rebase it onto the latest main.

Task 9: Undoing Changes

```
1. Make a new commit and undo it using soft reset.
2. Commit again and make another commit.
3. Undo it using mixed reset.
4. Commit again and make one more commit.
5. Undo it using hard reset.
6. Create a commit and use git revert to reverse it.
```

Task 10: Final Cleanup

```
1. Merge feature-dashboard into main.
2. Create an annotated tag v2.0.
3. View the complete Git history using:
git log --oneline --graph --all
4. Delete all merged feature branches safely.
```

Final Questions

```
1. Which symbol shows the currently active branch?
2. What is the difference between git branch -d and git branch -D?
3. When does a fast-forward merge occur?
4. When does a three-way merge occur?
5. Which command cancels an ongoing merge?
6. What is the difference between a lightweight tag and an annotated tag?
7. What does HEAD~1 mean?
8. What happens when you run git rebase main?
9. What is the difference between git reset and git revert?
10. What is the difference between soft, mixed, and hard reset?
```

Exercise 2: Complete Git Practice Challenge — Answer

Task 1: Initialize Repository

Create the project folder and initialize Git:

mkdir student-management-system

cd student-management-system

git init

Create the README.md file:

echo "Student Management System" > README.md

Add and commit the file:

git add README.md

git commit -m "Initial project setup"

Task 2: Student Feature

Create and switch to the feature-student branch:

git switch -c feature-student

Create students.txt:

echo "Student Registration Module" > students.txt

Add and commit the changes:

git add students.txt

git commit -m "Add student module"

Task 3: Course Feature

Switch back to main:

git switch main

Create and switch to the feature-course branch:

git switch -c feature-course

Create courses.txt:

echo "Course Management Module" > courses.txt

Add and commit the changes:

git add courses.txt

git commit -m "Add course module"

Task 4: Result Feature

Switch back to main:

git switch main

Create and switch to the feature-result branch:

git switch -c feature-result

Create results.txt:

echo "Student Result Module" > results.txt

Add and commit the changes:

git add results.txt

git commit -m "Add result module"

Task 5: Merge All Branches

Switch to the main branch:

git switch main

Merge the student feature:

git merge feature-student

Merge the course feature:

git merge feature-course

Merge the result feature:

git merge feature-result

After merging, the project should contain:

README.md

students.txt

courses.txt

results.txt

Task 6: Create and Resolve a Merge Conflict

First, make sure you are on main:

git switch main

Create message.txt:

echo "Welcome Students" > message.txt

Commit it:

git add message.txt

git commit -m "Add welcome message"

Create and switch to feature-message:

git switch -c feature-message

Change the file:

echo "Welcome Developers" > message.txt

Commit the change:

git add message.txt

git commit -m "Update message for developers"

Switch back to main:

git switch main

Change the same line differently:

echo "Welcome to Git Training" > message.txt

Commit the change:

git add message.txt

git commit -m "Update message on main"

Now merge the feature branch:

git merge feature-message

Git may show a conflict like this:

<<<<<<< HEAD

Welcome to Git Training

=======

Welcome Developers

>>>>>>> feature-message

Resolve the conflict by editing message.txt and keeping the following final content:

Welcome to Git Training and Development

Then add and commit the resolved file:

git add message.txt

git commit -m "Resolve merge conflict"

Task 7: Create Tags

Create a Lightweight Tag

git tag v1.0

Create an Annotated Tag

git tag -a v1.1 -m "Version 1.1 release"

View All Tags

git tag

Push a Single Tag

git push origin v1.1

Push All Tags

git push origin --tags

Note: The push commands require a remote repository named origin.

Task 8: Rebase

Create and switch to the feature-dashboard branch:

git switch -c feature-dashboard

Create dashboard.txt:

echo "Student Dashboard Module" > dashboard.txt

Add and commit:

git add dashboard.txt

git commit -m "Add dashboard module"

Switch to main:

git switch main

Make another change on main. For example:

echo "Development Team" > team.txt

Add and commit it:

git add team.txt

git commit -m "Add development team"

Switch back to the dashboard branch:

git switch feature-dashboard

Rebase it onto the latest main:

git rebase main

After rebasing, the history will conceptually look like:

A---B---C---D'

↑

feature-dashboard

The feature branch commits are replayed on top of the latest main.

Task 9: Undoing Changes

Step 1: Create a New Commit

For example:

echo "Dashboard Version 2" >> dashboard.txt

git add dashboard.txt

git commit -m "Update dashboard version 2"

Step 2: Undo Using Soft Reset

This removes the latest commit but keeps the changes staged:

git reset --soft HEAD~1

Now commit again:

git commit -m "Update dashboard version 2"

Step 3: Create Another Commit

echo "Dashboard Version 3" >> dashboard.txt

git add dashboard.txt

git commit -m "Update dashboard version 3"

Step 4: Undo Using Mixed Reset

This removes the latest commit but keeps the changes in the file as unstaged changes:

git reset --mixed HEAD~1

Add and commit again:

git add dashboard.txt

git commit -m "Update dashboard version 3"

Step 5: Create Another Commit

echo "Dashboard Version 4" >> dashboard.txt

git add dashboard.txt

git commit -m "Update dashboard version 4"

Step 6: Undo Using Hard Reset

This removes the latest commit and discards its changes:

git reset --hard HEAD~1

Be careful: --hard permanently discards uncommitted changes associated with that reset.

Step 7: Use Git Revert

Create another commit:

echo "Temporary Dashboard Feature" >> dashboard.txt

git add dashboard.txt

git commit -m "Add temporary dashboard feature"

Check the commit history:

git log --oneline

Copy the commit ID of the latest commit.

For example:

git revert <commit-id>

Or, if you want to revert the latest commit:

git revert HEAD

Git will create a **new commit** that reverses the changes.

Task 10: Final Cleanup

Step 1: Switch to Main

git switch main

Step 2: Merge the Dashboard Branch

git merge feature-dashboard

Step 3: Create an Annotated Tag

git tag -a v2.0 -m "Version 2.0 release"

Step 4: View Complete Git History

git log --oneline --graph --all

Step 5: Delete All Merged Feature Branches

```
git branch -d feature-student

git branch -d feature-course

git branch -d feature-result

git branch -d feature-message

git branch -d feature-dashboard
```

Final Expected Project Structure

After completing all tasks, your project should look similar to:

```
student-management-system/
|
 ├── README.md
 ├── students.txt
 ├── courses.txt
 ├── results.txt
 ├── message.txt
 ├── dashboard.txt
 └── team.txt
```

Important Commands Used

```
# Create and switch to a branch

git switch -c branch-name


# Switch branch

git switch branch-name


# Merge a branch

git merge branch-name
```

```
# Cancel a merge

git merge --abort


# Create a lightweight tag

git tag v1.0


# Create an annotated tag

git tag -a v1.1 -m "Message"


# Rebase

git rebase main


# Soft reset

git reset --soft HEAD~1


# Mixed reset

git reset --mixed HEAD~1


# Hard reset

git reset --hard HEAD~1


# Revert a commit

git revert <commit-id>


# View Git history

git log --oneline --graph --all
```

Delete a branch safely

git branch -d branch-name

Final Questions – Answers

1. Which symbol shows the currently active branch?

The * symbol shows the currently active branch.

Example:

feature-login

* main

feature-dashboard

Here, main is the currently active branch.

2. What is the difference between git branch -d and git branch -D?

git branch -d branch-name

- Safely deletes a branch.
- Git will prevent deletion if the branch contains unmerged changes.

git branch -D branch-name

- Forcefully deletes a branch.
- It deletes the branch even if its changes have not been merged.

3. When does a fast-forward merge occur?

A **fast-forward merge** occurs when the target branch has not changed since the feature branch was created.

For example:

main: A---B

\

feature: C---D

After merging:

main: A---B---C---D

Git simply moves the main branch pointer forward.

---

4. When does a three-way merge occur?

A **three-way merge** occurs when both branches have new commits after they diverge.

Example:

```
      C---D  feature
     /
A---B
     \
      E---F  main
```

Git uses the common ancestor and the changes from both branches to create a merge commit.

---

5. Which command cancels an ongoing merge?

Use:

git merge --abort

This cancels the merge and returns the repository to its state before the merge started.

---

6. What is the difference between a lightweight tag and an annotated tag?

A **lightweight tag** is simply a name that points to a specific commit.

git tag v1.0

An **annotated tag** contains additional information such as a tag message, tagger name, email, and date.

git tag -a v1.0 -m "Version 1.0 release"

---

7. What does HEAD~1 mean?

HEAD~1 refers to the **parent of the current commit**.

For example:

A---B---C

↑

HEAD

HEAD~1 refers to commit B.

---

8. What happens when you run git rebase main?

When you run:

git rebase main

Git takes the commits from your current branch and replays them on top of the latest main branch.

Example:

Before:

```
      C---D  feature
     /
A---B---E---F  main
```

After rebasing:

```
A---B---E---F---C'---D'
                ↑
            feature
```

The commits C and D are recreated as C' and D', so their commit IDs may change.

---

9. What is the difference between git reset and git revert?

git reset

git reset moves the branch pointer backward and can modify the staging area and working directory.

Example:

git reset --soft HEAD~1

It is generally useful for undoing local commits.

git revert

git revert does not remove the original commit. Instead, it creates a **new commit** that reverses the changes.

Example:

git revert <commit-id>

It is generally safer when working with commits that have already been shared with others.

10. What is the difference between soft, mixed, and hard reset?

Reset Type	Commit History	Staging Area	Working Directory
--soft	Removes commit	Changes remain staged	Changes remain
--mixed	Removes commit	Changes become unstaged	Changes remain
--hard	Removes commit	Changes removed	Changes discarded

Soft Reset

git reset --soft HEAD

The commit is removed, but changes remain **staged**.

Mixed Reset

git reset --mixed HEAD

The commit is removed, and changes remain in the working directory but become **unstaged**.

Hard Reset

git reset --hard HEAD

The commit is removed, and the changes are also discarded from the staging area and working directory.

Important: Be careful when using git reset --hard because discarded changes may be difficult to recover.